

Fragments Microservice — Technical Report

Assignment 2: API Server Checklist

Harsh Pahurkar

Course: CCP554 — Semester 6

Date: November 15, 2025

Table of Contents

1. Introduction
2. Repository and CI links
3. Implementation summary
4. Evidence & Screenshots
5. How to reproduce & verify (step-by-step)
6. Conclusion and outstanding work

1. Introduction

This report documents the work completed for Assignment 2: API Server Checklist for the Fragments microservice. It builds on the code delivered in Assignment 1 and adds support for:

- POST handling for text/* and application/json payloads
- GET /v1/fragments?expand=1 returning expanded metadata
- GET /v1/fragments/:id/info returning fragment metadata
- GET /v1/fragments/:id.ext supporting markdown (.md) → HTML (.html) via markdown-it
- Multi-stage Dockerfile and CI pipeline that builds and pushes Docker images
- Unit tests and coverage

2. Repository and CI links

Private GitHub repo for fragments: <https://github.com/harshpahurkar/fragments>

Private GitHub repo for fragments: <https://github.com/harshpahurkar/fragments-ui>

Public Docker Hub repository for Backend:

<https://hub.docker.com/repository/docker/harshpahurkar/fragments>

harshpahurkar
Docker Personal

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories / fragments / General

harshpahurkar/fragments

Last pushed less than a minute ago · Repository size: 485.2 MB · ☆0 · ↓190

Add a description

Add a category

General

Tags

Image Management

BETA

Collaborators

Webhooks

S

Tags

DOCKER SCOUT INACTIVE
[Activate](#)

This repository contains 7 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	less than a minute
main		Image	less than 1 day	8 days
sha-9b9d58f3ff9e75b...		Image	less than 1 day	8 days
sha-f9d2dcc6142975...		Image	8 days	8 days
sha-a60359471a7d5...		Image	8 days	8 days

[See all](#)

Repository overview INCOMPLETE

An overview describes what your image does and how to run it. It displays in [the public view of your repository](#) once you have pushed some content.

Add overview

Public Docker Hub repository for UI:

<https://hub.docker.com/repository/docker/harshpahurkar/fragments-ui/general>

The screenshot shows the Docker Hub repository page for **harshpahurkar/fragments-ui**. The repository is public and has 0 stars and 41 downloads. It was last pushed 1 day ago. The repository size is 20.2 MB. The page includes a sidebar with navigation options like Repositories, Hardened Images, Collaborations, Settings, Billing, and Usage. The main content area shows the repository details, including a table of tags. The 'latest' tag is the only one listed, pushed less than 1 day ago. There is also a section for 'Repository overview' which is currently incomplete. On the right, there are Docker commands and a section for Docker Build Cloud.

harshpahurkar
Docker Personal

Repositories / fragments-ui / General

harshpahurkar/fragments-ui

Last pushed 1 day ago · Repository size: 20.2 MB · ☆0 · ↓41

Add a description ⓘ

Add a category ⓘ

General Tags Image Management BETA Collaborators Webhooks Settings

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest	linux	Image	less than 1 day	1 day

[See all](#)

Repository overview ⓘ INCOMPLETE

An overview describes what your image does and how to run it. It displays in [the public view of your repository](#) once you have pushed some content.

[Add overview](#)

Using 0 of 1 private repositories.

Docker commands

To push a new tag to this repository:

```
docker push harshpahurkar/fragments-ui:tagname
```

[Public view](#)

buildcloud

Build with
Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

[Go to Docker Build Cloud](#) →

GitHub Actions CI (example run):

The screenshot shows the GitHub Actions CI workflow runs page. The left sidebar has a 'New workflow' button and a list of workflow runs. The main content area shows a list of workflow runs. The first run is 'ci: trigger build for submission' (ci #17) which was pushed by harshpahurkar on the main branch. It was completed 2 minutes ago. The second run is 'Release v0.7.0' (ci #16) which was pushed by harshpahurkar on the main branch. It was completed on Nov 7, 6:34 PM EST. The third run is 'Release v0.7.0' (cd #3) which was pushed by harshpahurkar on the v0.7.0 branch. It was completed on Nov 7, 6:34 PM EST.

Actions

New workflow

All workflows

Showing runs from all workflows

Filter workflow runs

Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

[Give feedback](#)

20 workflow runs

Event Status Branch Actor

ci: trigger build for submission

ci #17: Commit 5dd832f pushed by harshpahurkar

main

2 minutes ago

1m 16s

Release v0.7.0

ci #16: Commit 9b9d58f pushed by harshpahurkar

main

Nov 7, 6:34 PM EST

1m 12s

Release v0.7.0

cd #3: Commit 9b9d58f pushed by harshpahurkar

v0.7.0

Nov 7, 6:34 PM EST

51s

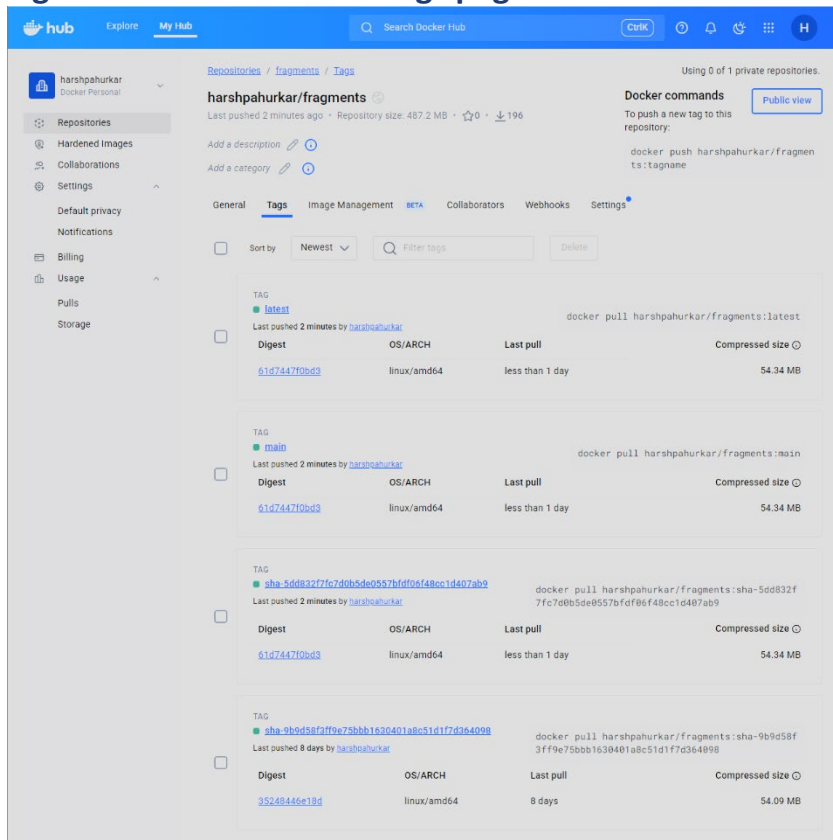
3. Implementation summary

Key files added/modified:

- `src/routes/api/post.js` — Accepts `application/json` and `text/*`, normalizes JSON before storage.
- `src/routes/api/get.js` — Supports `?expand=1` by forwarding to the model.
- `src/routes/api/getById.js` — Returns content with appropriate Content-Type. If stored JSON is corrupted, returns raw content as `text/plain`.
- `src/routes/api/getInfo.js` — Returns metadata for a fragment.
- `src/routes/api/getByIdExt.js` — Supports `id.html` converting markdown to HTML via `markdown-it`.
- `src/model/fragments.js` — Added expand behavior and exported `getFragmentMeta`.
- `Dockerfile` — Multi-stage Dockerfile producing a small runtime image.
- `.github/workflows/ci.yml` — CI pipeline: lint, unit-tests+coverage, build & push image to Docker Hub.

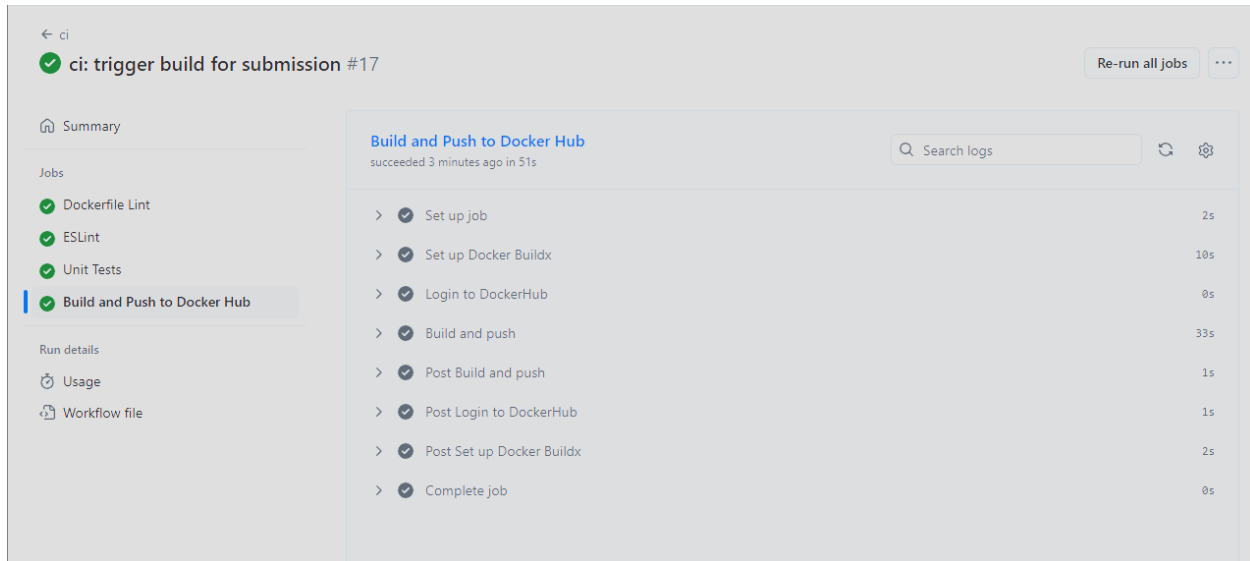
4. Evidence & Screenshots

Figure 4.1 — Docker Hub tags page



Latest Tags Page

Figure 4.2 — GitHub Actions CI run showing successful docker push



Docker Build and Push working

Figure 4.3 — EC2: running container and server logs

```
[ec2-user@ip-172-31-29-247 package]$ docker ps --filter name=fragments
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
f579552adc91   harshpahurkar/fragments:latest      "docker-entrypoint.s..." 32 minutes ago Up 32 minutes 0.0.0.0:8080->8080/tcp, :
:8080->8080/tcp   fragments
[ec2-user@ip-172-31-29-247 package]$
```

Docker ps showing

```
[ec2-user@ip-172-31-29-247 package]$ docker logs -f fragments
[dotenv@17.2.2] injecting env (0) from .env -- tip: $ run anywhere with `dotenvx run -- yourcommand`
[22:54:36.361] INFO (1): Using AWS Cognito for auth
[22:54:36.470] INFO (1): Server started on port 8080
[22:54:36.550] INFO (1): Cognito JWKS cached
```

Localhost 8080 running on docker

```

Unauthorized[ec2-user@ip-172-31-29-247 ~]$ curl -i http://localhost:8080/
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancest
script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Access-Control-Allow-Origin: *
X-App-Version: 0.7.0
Cache-Control: no-cache
Content-Type: application/json; charset=utf-8
Content-Length: 116
ETag: W/"74-pDAF+T6HwtHwli95rMXufEP8uzo"
Vary: Accept-Encoding
Date: Sun, 16 Nov 2025 22:59:35 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"ok","author":"HARSH PAHURKAR","githubUrl":"https://github.com/harshpahurkar/fragments","version":"0.7.0"}[ec2-

```

Localhost 8080 curl running

Figure 4.4 — fragments-ui + Cognito

The screenshot shows a web browser at `localhost:1234/callback` displaying the 'Fragments UI' application. The application has a 'Login' button and a 'Logout' button. Below the buttons, it says 'Status: No fragments found'. A greeting 'Hello harshpahurkar!' is shown. There is a 'Create a new fragment' section with a 'text/plain' dropdown and a 'Create Fragment' button. Below that, it says 'Your fragments (no fragments)'. The browser's developer tools are open, showing the Network tab with a list of requests and the Console tab with a log of the application's state and API calls.

Name	Status	Type	Initiator	Size
bamboo.min.css	200	stylesheet	bamboo.css	1...
data:image/svg+xml...	200	svg+xml	bamboo.min.css	(memory cache) 0 ...
fragments	204	preflight	Preflight	0.0 kB 8...
localhost	101	websocket	fragments-ui.077c784.js:84	0.0 kB 2...
sidePanelUtils.js	200	script	GenAIWebpageEligibilityService.js:18	1.6 kB 5...
readability.js	200	script	sidePanelUtils.js:18	94.0 kB 2...
favicon.ico	304	text/html	Other	0.3 kB 5...
fragments	200	fetch	api.js:60	1.0 kB 6...
fragments?expand=1	200	fetch	api.js:60	1.0 kB 4...
fragments?expand=1	204	preflight	Preflight	0.0 kB 3...

Console Log:

```

User Authenticated {user: _User}
STATUS: Authenticated as harshpahurkar
getApiBase: using meta api-url http://ec2-3-84-104-142.compute-1.amazonaws.com:8080
authorizationHeaders: choosing token (preferring id_token)
Document already loaded, running initialization immediately
Attempting to initialize Adunit
Adunit initialized successfully
GET /v1/fragments ok {status: 'ok', fragments: Array(0)}
fragments {status: 'ok', fragments: Array(0)}
STATUS: Loading fragments...
getApiBase: using meta api-url http://ec2-3-84-104-142.compute-1.amazonaws.com:8080
authorizationHeaders: choosing token (preferring id_token)
GET /v1/fragments ok {status: 'ok', fragments: Array(0)}
STATUS: No fragments found

```

UI Running locally and connecting on EC2

Your fragments

id: 98330d97-f2d6-42c0-b6e9-6cfec2665824
owner: f3eea0a5e89b7b86d7ae43a46c3994a6b9b8d2fa9d970fe721e7b5a04dc5e9b3
contentType: text/plain
size: 13
created: 2025-11-17T01:01:11.522Z

id: 5d546736-7bec-4816-bc9b-9659a2c20239
owner: f3eea0a5e89b7b86d7ae43a46c3994a6b9b8d2fa9d970fe721e7b5a04dc5e9b3
contentType: text/markdown
size: 25
created: 2025-11-17T01:04:02.658Z

id: e50c90c6-0b01-47cb-a841-b03dde3c6144
owner: f3eea0a5e89b7b86d7ae43a46c3994a6b9b8d2fa9d970fe721e7b5a04dc5e9b3
contentType: application/json
size: 31
created: 2025-11-17T01:04:23.684Z

id: 1b3c7877-27e6-4ab0-a471-0c07348edc4d

STATUS: Checking authentication...
Document already loaded, running initialization immediately
Attempting to initialize Adunit
Adunit initialized successfully
User Authenticated {user: e}
STATUS: Authenticated as harshpapurkar
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
GET /v1/fragments ok {status: 'ok', fragments: Array(4)}
fragments: (4) [{"98330d97-f2d6-42c0-b6e9-6cfec2665824", "5d546736-7bec-4816-bc9b-9659a2c20239", "e50c90c6-0b01-47cb-a841-b03dde3c6144", "1b3c7877-27e6-4ab0-a471-0c07348edc4d"}]
status: "ok"
[[Prototype]]: Object
fragments {status: 'ok', fragments: Array(4)}
fragments: (4) [{"98330d97-f2d6-42c0-b6e9-6cfec2665824", "5d546736-7bec-4816-bc9b-9659a2c20239", "e50c90c6-0b01-47cb-a841-b03dde3c6144", "1b3c7877-27e6-4ab0-a471-0c07348edc4d"}]
status: "ok"
[[Prototype]]: Object
STATUS: Loading fragments...
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
GET /v1/fragments ok {status: 'ok', fragments: Array(4)}
STATUS: Loaded 4 fragments
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
POST /v1/fragments ok {status: 'ok', fragment: [-]}
Created fragment: {data: [-], headers: [-]}
data: {status: 'ok', fragment: [-]}
headers: {location: 'http://ec2-3-84-104-142.compute-1.amazonaws.com:80..v1/fragments/2660b276-5404-4ff3-844b-03623b31f740'}
[[Prototype]]: Object
STATUS: Loading fragments...
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
GET /v1/fragments ok {status: 'ok', fragments: Array(5)}
STATUS: Loaded 5 fragments

Markdown and JSON fragment created with them being shown in the console along with location header set and returned correctly. This shows metadata for existing fragments aswell

localhost:1234/callback

contentype: text/plain
size: 11
created: 2025-11-17T01:05:03.587Z

id: 2d60b276-5404-4ff3-844b-03623b31f740
owner: f3eea0a5e89b7b86d7ae43a46c3994a6b9b8d2fa9d970fe721e7b5a04dc5e9b3
contentype: text/plain
size: 17
created: 2025-11-17T01:07:34.697Z

id: 5d4a5291-955a-48fc-96de-64b62b94945c
owner: f3eea0a5e89b7b86d7ae43a46c3994a6b9b8d2fa9d970fe721e7b5a04dc5e9b3
contentype: text/markdown
size: 21
created: 2025-11-17T01:22:51.173Z

id: 90b8ba79-409f-4001-839d-82eb6d45cd7c
owner: f3eea0a5e89b7b86d7ae43a46c3994a6b9b8d2fa9d970fe721e7b5a04dc5e9b3
contentype: application/json
size: 33
created: 2025-11-17T01:23:37.538Z

STATUS: Loading fragments...
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
GET /v1/fragments ok {status: 'ok', fragments: Array(5)}
STATUS: Loaded 5 fragments
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
POST /v1/fragments ok {status: 'ok', fragment: [-]}
Created fragment: {data: [-], headers: [-]}
data: {status: 'ok', fragment: [-]}
headers: {location: 'http://ec2-3-84-104-142.compute-1.amazonaws.com:80..v1/fragments/5d4a5291-955a-48fc-96de-64b62b94945c'}
[[Prototype]]: Object
STATUS: Loading fragments...
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
GET /v1/fragments ok {status: 'ok', fragments: Array(6)}
STATUS: Loaded 6 fragments
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
POST /v1/fragments ok {status: 'ok', fragment: [-]}
Created fragment: {data: [-], headers: [-]}
data: {status: 'ok', fragment: [-]}
headers: {location: 'http://ec2-3-84-104-142.compute-1.amazonaws.com:80..v1/fragments/90b8ba79-409f-4001-839d-82eb6d45cd7c'}
[[Prototype]]: Object
STATUS: Loading fragments...
getApiBase: using meta api-url <http://ec2-3-84-104-142.compute-1.amazonaws.com:8080>
authorizationHeaders: choosing token (preferring id_token)
GET /v1/fragments ok {status: 'ok', fragments: Array(7)}
STATUS: Loaded 7 fragments

New JSON and markdown created realtime with their headers shown aswell

Figure 4.5 — Tests & coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.82	64	97.5	95.22	
src	100	78.57	100	100	
app.js	100	66.66	100	100	42-43
hash.js	100	100	100	100	
logger.js	100	75	100	100	4
response.js	100	100	100	100	
src/auth	91.07	80	90.9	90.38	
auth-middleware.js	100	100	100	100	
basic-auth.js	100	100	100	100	
cognito.js	85.71	100	80	85	41,56-57
hash-user.js	100	50	100	100	6
index.js	71.42	84.61	100	71.42	12,27
src/model	96.87	70.58	100	100	
fragment.js	90.9	77.77	100	100	24-27
fragments.js	100	62.5	100	100	6-13
src/model/data	100	100	100	100	
index.js	100	100	100	100	
src/model/data/memory	100	80.76	100	100	
index.js	100	100	100	100	
memory-db.js	100	80.76	100	100	21,34,41,47
src/routes	100	100	100	100	
index.js	100	100	100	100	
src/routes/api	91.42	51.61	100	92.15	
get.js	100	63.63	100	100	11-12
getById.js	95.23	44.44	100	95	38
getByIdExt.js	95.23	52.38	100	100	8-13,18,32
getInfo.js	100	30	100	100	8-9
index.js	92.85	50	100	92.85	19
post.js	81.81	58.06	100	81.81	13,29-34
Test Suites: 27 passed, 27 total					
Tests: 48 passed, 48 total					
Snapshots: 0 total					
Time: 3.02 s					
Ran all test suites.					

Showing npm coverage above 80

5. How to reproduce & verify (concise checklist)

This section gives a concise, step-by-step guide a grader can follow to verify the API checklist items and evidence.

Preparation

1. Clone the repo (private instructor repo or your private link):
2. `git clone https://github.com/harshpahurkar/fragments.git`
3. `cd fragments`
4. Install dependencies and run unit tests locally:
5. `npm ci`
6. `npm test`
`npm run coverage`

Confirm tests pass and coverage summary is visible; screenshot it.

CI / Docker Hub verification

1. Open GitHub Actions → select the latest `ci` run → verify the unit-tests job passed and the docker build/push step completed. Screenshot the build-push logs.
2. Open Docker Hub (public URL) and confirm image tags (latest/main/sha) exist. Screenshot the tags page.

EC2 run & API verification

1. SSH to EC2 instance and run the commands under Figure 4.3 to pull & run the container.
2. Use curl to interact with health and API endpoints; screenshot responses showing headers and body. Example:
3. `curl -i http://localhost:8080/`
4. `curl -i -X POST -H "Content-Type: application/json" -u user1@email.com:password1 -d '{"a":1}' http://localhost:8080/v1/fragments`

6. Conclusion and outstanding work

Completed:

- All required API endpoints implemented and unit-tested.
- Multi-stage Dockerfile and CI workflow implemented to push images to Docker Hub.
- Unit tests and coverage implemented; statement/line coverage well above 80%.

Notes on known issues

- If stored JSON is corrupted, the server returns the raw content as `text/plain` (this avoids sending invalid JSON body with a JSON content-type). This behaviour is documented in the report and covered by a unit test.
 - The Dockerfile currently copies `tests/.htpasswd` into the build stage. It's included for test convenience; if you want a production-only image, remove test files from the final runtime image.